College of Computing and Informatics
Department of Machine Learning

Course Project Brief Catalog

ITAI 4160 / 5160: Databases and Cloud for Artificial Intelligence

Industry Project Preview | *October 28, 2026*

Instructor: Dr. Qiong Cheng (`qcheng1@charlotte.edu`)

We gratefully acknowledge **Wells Fargo** and **Ken Meyer, CIO**, for their executive leadership and participation in this Industry Advisory & Project Preview Session.

Catalog Index

ITAI 4160 / 5160: Databases and Cloud for Artificial Intelligence

Department of Machine Learning | College of Computing and Informatics | UNC Charlotte

Instructor: Dr. Qiong Cheng (qcheng1@charlotte.edu)

Part 1. Course Overview

Page 2

Course Brief: Databases and Cloud for Artificial Intelligence *Executive Overview*

Focus Areas: Databases, Cloud Infrastructure, Data Engineering, Distributed AI, Vector Search & RAG, LLM Applications, and Enterprise AI Architecture

Part 2. Team Project Briefs

Team ID	Project Name	Primary Focus / Domain	Page
Team 1	RAEJ	LLM Translation for Languages Without Parallel Data	6
Team 2	MCP SENTINEL	AI Security Intelligence for MCP	8
Team 3	Class Wizard	Intelligent Academic Scheduling & Course Planning Platform	10
Team 4	Financial Literacy	AI-Driven Personal Finance & Educational Analytics Engine	12
Team 5	FitWise AI	Personalized Gym Coaching & Algorithmic Workout Prescriptions	14
Team 6	S-Card	Personalized Spam Protection	16
Team 7	CoursePilot	Autonomous Academic Degree Navigation & Advisory Assistant	18

Industry Advisory & Project Preview Session | Executive Reviewer: **Ken Meyer**, CIO, Wells Fargo

Databases and Cloud for Artificial Intelligence

Academic Syllabus Overview | *Industry Advisory & Project Preview Session*

Executive Reviewer: Ken Meyer, CIO, Wells Fargo | Fall 2026 Course Overview

Course Name:	Databases and Cloud for Artificial Intelligence
Course Code:	ITAI 4160 & 5160 (Cross-listed Undergraduate & Graduate)
Term & Cohort:	Fall 2026 | 29 Students (41% Graduate, 38% Senior Undergrad, 10% Early Entry (Senior+Graduate), 10% Junior Undergrad)
Prerequisites:	Python Programming (ITSC 1213) (C or Above), Computer Systems (ITSC 2181) (C or Above)

1. Course Description

Databases and Cloud for AI introduces the fundamental concepts, technologies, and system architectures used to build modern AI applications on scalable data and cloud platforms. Students learn how structured, semi-structured, and unstructured data are collected, stored, processed, and managed throughout the AI data lifecycle.

The course explores the end-to-end AI pipeline, from data acquisition, wrangling, and database design to distributed data processing, model inference, information retrieval, and AI-powered applications. It emphasizes how databases, cloud computing, and data engineering work together to support reliable, scalable, secure, and maintainable AI systems.

Through hands-on projects, students gain experience designing cloud-based data pipelines, managing relational and NoSQL databases, processing large-scale data, and integrating data infrastructure with modern AI workflows. Students develop the ability to design end-to-end AI system architectures, evaluate architectural trade-offs, and understand how data and cloud infrastructure enable the development and deployment of intelligent applications in enterprise environments.

2. Learning Objectives

By the end of the course, students will be able to:

- **Explain** how databases, cloud computing, and distributed data storage and processing work together to support modern AI applications.
- **Evaluate and select** appropriate data models and database technologies, based on data characteristics, AI workload requirements, and performance trade-offs.
- **Prepare, transform, and manage** structured, semi-structured, and unstructured data to support analytics, machine learning, and AI workflows.
- **Design and implement** data pipelines that integrate databases, distributed storage, and cloud technologies to support scalable, secure, and reliable data management.
- **Apply** distributed data processing techniques, including parallel processing, query execution, Apache Spark, and ETL workflows, to efficiently process and analyze large-scale datasets.
- **Analyze, design, and communicate** end-to-end AI data architectures that integrate data acquisition, storage, processing, retrieval, and AI model inference, while **collaborating effectively** in a team-based project.

3. Student Cohort Demographics & Technical Profile

The cross-listed **ITAI 4160 / 5160** course brings together 29 undergraduate and graduate students from a range of computing-related disciplines. The following breakdown provides an overview of students fields of study, academic standing, and course engagement, illustrating the range of backgrounds and curricular interests represented in the cohort (see Figure 1).

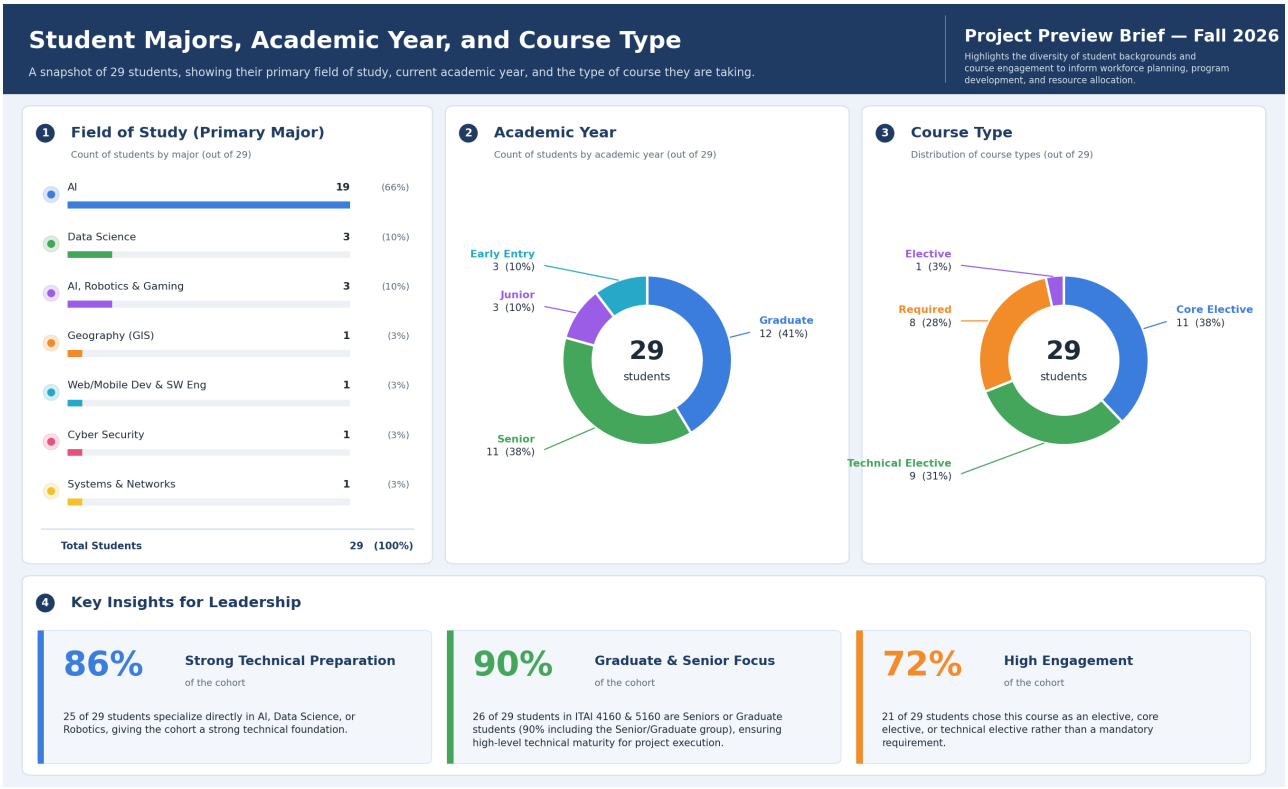


Figure 1: Student Cohort Profile — Field of Study, Academic Standing, and Enrollment Type.

4. Course Technical Scope & Core Infrastructure Overview

Table 1: **Technical Scope & Core Infrastructure Overview**

Data Platforms	PostgreSQL, pgvector, MongoDB, Neo4j, AWS S3, HDFS
Distributed Computing & Cloud	Apache Spark (PySpark, MLLib), Hadoop, AWS EC2, Docker, Kubernetes (K3s), Hybrid Cloud
Serverless & Event-Driven	AWS Lambda, Event-Driven Architectures, Serverless Data Pipelines
Application Development	Python, SQL, FastAPI, REST APIs
AI & ML Libraries	Scikit-learn, spaCy, Feature Engineering, Model Inference
LLM Application Frameworks	Ollama, LangChain, LlamaIndex
Applied AI Architectures	Vector Search, Retrieval-Augmented Generation (RAG), Model Context Protocol (MCP), Agentic Workflows

5. Project Milestones & Deliverables Timeline

Table 2: Project Roadmap: From Business Problem to AI-Enabled Solution

Phase / Milestone	Focus	Expected Deliverable	CIO Checkpoint
Proposal Sep. 12	Problem definition, business requirements, and system architecture	Defined use case, requirements, and proposed architecture	Project scope and value proposition
Sprint 1 Oct. 3	Data exploration and wrangling	Explored, assessed, and prepared datasets	Data readiness
Sprint 2 Oct. 22	Database architecture and implementation	Data models and core database infrastructure	Data management foundation
Sprint 3 Oct. 31	LLM-powered querying, chatbot development, and RAG indexing and retrieval	Initial AI application integrating LLM queries, conversational interaction, and retrieval from project data	AI-enabled data access (Initial)
Project Preview Oct. 28	**Phase 1: Architecture and core functionality**	Core databases, data pipelines, and initial AI application capabilities, as applicable to each team	**Initial framework (or working-prototype) demonstration**
Sprint 4 Nov. 14	Distributed data processing with Apache Spark and machine learning with Spark MLlib	Spark-based data processing workflows and, where appropriate, distributed machine-learning models	Scalable analytics and ML
Sprint 5 Nov. 22	End-to-end integration, deployment, and system evaluation	Integrated application, testing, and evaluation results	System readiness
Final Showcase Nov. 30	**Phase 2: End-to-end integration and evaluation**	Integrated application, AI capabilities, results, and final demonstration, as applicable to each team	**Final system demonstration**

Note: The Oct 28 preview emphasizes architecture and working core functionality; the Nov 30 showcase emphasizes end-to-end integration, evaluation, and application readiness.

- **Oct 28 Preview Deliverable:** Working prototype demonstration & core architecture validation.
- **Nov 30 Showcase Deliverable:** End-to-end integrated application with benchmark evaluations.

6. Context for Executive Reviewers

To help guide your feedback during student interactions, the following areas highlight where industry perspectives can provide the greatest value.

1. Architectural Decisions & Trade-Offs

Students have designed and implemented data pipelines, databases, and initial frameworks of AI-enabled applications. Feedback on how to evaluate and justify architectural choices—balancing performance, scalability, cost, reliability, security, and maintainability—would help students understand how technical decisions align with enterprise requirements. Examples include choosing between batch and real-time processing, relational and NoSQL databases, and traditional query-based systems and retrieval-augmented generation (RAG).

2. Production Governance & Business Impact

Students benefit from understanding how enterprise leaders evaluate AI-enabled systems beyond technical functionality, including data privacy, security, reliability, model performance, operational risk, regulatory requirements, and measurable business value. Industry perspectives can help students connect system evaluation to business priorities and responsible deployment.

Team 1: RAEJ — LLM Translation for Languages Without Parallel Data

Project Title:	RAEJ: LLM Translation for Languages Without Parallel Data
Team Members:	Alec Pike, Ryan Brown, Eric Medlock (Team Leader), Justin MacCreery
Domain / Industry:	NLP / Language AI (Machine Translation)

1. Business Problem & Target Impact

Machine translation depends on parallel text, leaving most of the worlds 7,000 languages, and all constructed languages, unsupported, though many have a dictionary and grammar. RAEJ tests whether an LLM can translate reliably from those references alone, using retrieval instead of training data, a pattern that also fits enterprises translating with controlled vocabularies they cannot train on. Impact metrics: gold-set accuracy, token cost per translation, and an audit trail of the entries and rules behind each output.

2. Technical Scope & Architecture

Data Sources: Yopowi conlang dictionary (3,015 entries, 4,402 meanings) and grammar (68 rules) from Vulgar Lang; team-validated gold test set.

Methodology: spaCy lemmatization; RAG over pgvector semantic search (Nomic Embed, one vector per meaning) plus SQL grammar lookup; tiered OOV fallback; PySpark Bronze/Silver/Gold pipeline; full-context vs. RAG evaluation (exact match, chrF, round-trip).

Primary Tech Stack:
Python, LangChain, PostgreSQL + pgvector, LM Studio on a self-hosted M4 Mac mini (Tailscale Funnel), Ollama, OpenAI API, AWS EC2, Docker Compose.

3. Project Milestones & Deliverables

Current Progress	Dictionary and grammar generated, PostgreSQL + pgvector schema deployed, lemma normalization built. Closing Sprint 2 (Oct 22): ingestion, embeddings, gold test set, OOV strategy.
Oct 28 Preview Deliverable	Live semantic lookup on the loaded lexicon (English word ranked Yopowi candidates) plus one English -> Yopowi translation from our reference prototype.
Nov 30 Showcase Deliverable	Bidirectional web app that shows its work, with RAG vs. full-context results on accuracy and token cost, packaged in Docker Compose.

4. Key Discussion Questions for Executive Reviewers

We welcome feedback on two core technical and strategic governance challenges:

1. Technical / Architectural Scaling

With context windows expanding into millions of tokens, when does RAG remain necessary over long-context prompting? Beyond token capacity, how do you evaluate latency, inference cost, and retrieval accuracy? *(Note: Our dataset spans ∼28k tokens.)*

2. Business / Governance & Compliance

In a regulated enterprise, where do you draw the line between self- hosted models and cloud LLM APIs on cost, privacy and control?

Team 2: MCP SENTINEL — AI-Powered Security Intelligence for MCP Servers

Project Title:	From MCP Server Discovery to Explainable Security Risk Intelligence
Team Members:	Sana Shaik, Sahir Shaik, Shreem Dixit, Dias Utebekov (Team Leader)
Domain / Industry:	AI Security, Cybersecurity, and Enterprise Technology Risk

1. Business Problem & Target Impact

An AI agent can be trusted based on its access to the tools it can use. With the help of the Model Context Protocol (MCP), an AI agent can access databases, files, APIs, and business applications. Yet, an MCP server can pose a security risk through vulnerabilities in its dependencies, exposure of its credentials, inadequate authentication, unnecessary permissions, or an alteration in its tool description. For a company like Wells Fargo, the problem does not lie merely in finding out about a vulnerable package. The issue is about knowing which MCP servers are used, what they are connected to, what security signals they provide, and whether there is any change in the risk they pose. MCP Sentinel can solve this problem by providing visibility into the situation. MCP Sentinel will be able to tie together the metadata of a server, information about the packages and dependencies it uses, vulnerabilities and historical data from scans.

Expected impact on the target: Increased asset visibility, increased security review priority, early identification of significant changes, and decreased investigation efforts for connected AI tools. Such impacts will have to be verified via testing because this project currently does not claim any measurable reduction in incidents or investigation times.

2. Technical Scope & Architecture

Data Sources: DeepNLP MCP Servers from Hugging Face for initial server identification; MCP Official Server registry for server data; GitHub API for repository data; npm & PyPI for package data; OSV.dev & deps.dev for vulnerability/dependency enrichment.

Methodology: Batch & API Ingestion -> Raw Data Storage -> Normalization & Deduplication -> Entity Resolution -> Package/Vulnerability Enrichment -> Feature Generation -> Explainable & Rule-Based Risk Scoring -> Scan Comparison.

Security Indicators: Severity of vulnerability, versions of affected packages, dependencies at risk, repository maintenance, and modifications to server or tool definitions. Authentication, authorization, and other risks specific to MCP will require proper data to allow for their assessment.

Primary Tech Stack: Proposed PostgreSQL database, possibly with JSONB for source metadata, processing and scoring in Python, and Docker for deployment. AWS is still a candidate for hosting, but not for deployment. Supervised learning is a future possibility, since there are no reliable risk labels at this time.

3. Project Milestones & Deliverables

Current Progress	Initial data sources, dataset attributes, data quality risks, processing pipeline, and proposed relational schema have all been identified.
Oct 28 Preview Deliverable	Introduce the security use case, the pipeline from sources to risks, the database schema, and any functional ingestion or scoring modules developed by the time of the class.
Nov 30 Showcase Deliverable	Aim to demonstrate an end-to-end solution for linked MCP server logs, explainable risk analysis, and historic change comparisons.

4. Key Discussion Questions for Industry Mentors

We welcome feedback on two core technical and strategic governance challenges:

1. Technical / Architectural Scaling

When an AI application uses tools provided by MCP servers, how should the system detect changes or inconsistencies in tool identities, versions, and definitions to maintain reliability, security, and compatibility?

2. Business / Governance & Compliance

What criteria should an enterprise security organization establish before using an MCP risk score to policy-approve, restrict, or ban an AI-connected tool?

Team 3: Class Wizard — Intelligent Academic Scheduling & Course Planning Platform

Project Title:	ClassWizard
Team Members:	Akshat Kumar, Caleb Richardson (Team Leader), Jackson Redman, Scott Do
Domain / Industry:	Educational AI

1. Business Problem & Target Impact

With the diverse classes offered at the university, college students struggle to find which classes best align with their career goals. Current tools like DegreeWorks track graduation requirements, but fail to guide students on which electives best align with their career path. Classwizard provides a solution through a career-aware, prerequisite-focused planning system. It achieves measurable business impact by improving student job readiness by suggesting the classes that teach the career-specific skills.

2. Technical Scope & Architecture

Data Sources: PostgreSQL structured course information + O*NET SQL career Database + PostgreSQL University Syllabus Repository + Redis DegreeWorks information.

Methodology: Batch ETL pipeline prep, spaCy, pgvector Semantic Search, RAG, NLP mode.

Primary Tech Stack: PostgreSQL, pgvector, Docker, AWS (EC2, S3, Lambda), FastAPI, Open-Source LLMs

3. Project Milestones & Deliverables

Current Progress	O*NET career database ETL complete, education ontology defined, and scaffolded knowledge graph pipeline.
Oct 28 Preview Deliverable	Live data pipeline demo O*NET career database loaded into Neo4j graph (occupations, skills, knowledge areas); UNC Charlotte course syllabi processed through LLM triple extraction pipeline using education ontology; working graph query demonstrating course-to-career skill alignment across both data sources.
Nov 30 Showcase Deliverable	End-to-end ClassWizard application career goal input triggers pgvector semantic search across UNC Charlotte course knowledge graph and O*NET career profiles; RAG pipeline generates grounded course recommendations with skill-match justification; deployed Streamlit interface with graph explorer, career-to-course query interface, and recommendation report view.

4. Key Discussion Questions for Industry Mentors

We welcome feedback on two core technical and strategic governance challenges:

1. Technical / Architectural Scaling

How can organizations protect sensitive data when integrating information from internal systems into data analytics and AI applications?

2. Business / Governance & Compliance

ClassWizards core assumption is that O*NETs occupational skill profiles accurately represent what employers actually want from university hires. As someone who recruits at scale, how closely does O*NETs skill taxonomy match what Wells Fargo finds missing in entry-level candidates and if theres a gap, is it in the skills O*NET defines, or in how universities teach toward them?

Team 4: Financial Literacy Service — AI-Driven Personal Finance & Educational Analytics Engine

Project Title:	Financial Literacy Service: AI-Powered Market Context for Beginner Investors
Team Members:	Josh Marsico (Team Leader), Jayden Parsons, Himaja Goda, Michael Simms
Domain / Industry:	Financial AI, Financial Literacy, and NLP

1. Business Problem & Target Impact

The project addresses the difficulty beginner investors face in understanding financial terminology, market trends, and current news. The enterprise pain point is the lack of a unified, accessible tool that connects financial data with relevant events and explains them in simple terms. This matters to investors seeking financial literacy and financial firms aiming to improve user engagement. The measurable impact includes reducing redundant API calls, improving data retrieval efficiency, minimizing duplicate articles, and providing relevant, citation-backed explanations.

2. Technical Scope & Architecture

Data Sources: Alpha Vantage historical market data, yfinance as a fallback for stock prices, and Tavily for financial news and web search.

Methodology: Data wrangling and normalization, API data ingestion, duplicate article detection, PostgreSQL persistence with upserts, market-event analysis, and citation-backed financial explanations using retrieval-augmented generation (RAG).

Primary Tech Stack: React, TypeScript, Recharts, Python 3.12, FastAPI, PostgreSQL 16, pgvector, OpenAI text-embedding-3-small, and Docker Compose. Redis and ontology/agent orchestration are proposed components unless implementation is confirmed.

3. Project Milestones & Deliverables

Current Progress	Sprint 1 established the problem scope, candidate data providers, stack, and initial architecture. Risks identified include duplicate news, overlapping symbol/date price records, provider rate limits, and missing article dates or content. Persistent database integration and modular ingestion in Sprint 2 are priorities; distinguish completed work from planned work in the live demo. Draft architectural representation can be found here.
Oct 28 Preview Deliverable	Current working pipeline and architecture, explain the selected data sources, and demonstrate any completed ingestion/normalization or database components.
Nov 30 Showcase Deliverable	End-to-end deployed application, interactive frontend dashboard, and full RAG pipeline.

4. Key Discussion Questions for Industry Mentors

We welcome feedback on two core technical and strategic governance challenges:

1. Technical / Architectural Scaling

What governance, audit logging, and human-review controls should be in place before an AI system presents financial-news explanations to customers?

2. Business / Governance & Compliance

How are you building trust in AI systems both internally and with consumers as you continue to integrate more? Do you see more people pushing back against AI systems in the finance space or is it more readily adopted?

Team 5: FitWise AI — Personalized Gym Coaching & Algorithmic Workout Prescriptions

Project Title:	FitWise AI — Personalized Gym Coaching & Algorithmic Workout Prescriptions
Team Members:	Josiah Duran (Team Leader), Raymundo Munoz Cuevas, Ali Chaaraoui (Team Leader), Essey Beraki
Domain / Industry:	Health & Fitness A

1. Business Problem & Target Impact

Most fitness apps hand users a generic program and a blank log, so they guess what to lift, which muscles they neglect, and whether they will reach their goal. FitWise AI learns from each users training, sleep, and activity to prescribe every split and weight, then forecasts the date they will hit their goal. Beginners get coach-level guidance; trainers and gyms see which members are stalling before they quit. Targets: next-lift predictions within 5% and plateaus flagged within two weeks of onset.

2. Technical Scope & Architecture

Data Sources: Fitbit data (35 users, 1,270 days of activity and sleep), free-exercise-db (876 exercises tagged by muscle), MegaGym, and simulated workout logs (19,736 sets)

Methodology: Each set is credited to the muscles it trains, building a weekly per-muscle ledger. An XGBoost model trained on all users, then fine-tuned per user, predicts each lifts next weight and flags plateaus; quantile regression gives the goal date a confidence band. An LLM with RAG explains each plan.

Primary Tech Stack: PostgreSQL + pgvector, PySpark (Bronze/Silver/Gold), FastAPI, React, Docker, AWS S3/EC2/RDS.

3. Project Milestones & Deliverables

Current Progress	Sprint 1 complete: all sources profiled and cleaned; every logged exercise links to the library. Sprint 2 (database) underway.
Oct 28 Preview Deliverable	Cleaned data in PostgreSQL and a first muscle ledger for a sample user.
Nov 30 Showcase Deliverable	A deployed app with weekly plans and target weights, a muscle body map, goal-date forecasts, and training-to-sleep insights.

4. Key Discussion Questions for Industry Mentors

We welcome feedback on two core technical and strategic governance challenges:

1. Technical / Architectural Scaling

When should an organization move from a unified data platform to specialized systems, considering the trade-offs among scalability, performance, cost, and operational complexity?

2. Business / Governance & Compliance

When an AI system provides forecasts with uncertainty ranges, how should organizations communicate those predictions so customers understand the uncertainty, trust the information appropriately, and avoid over-relying on the results?

Team 6: S-Card — Personalized Spam Protection

Project Title:	S-Card Personalized Spam Protection
Team Members:	Smriti Bhemireddy, Connor Kelly, Andrew Gallegos Garcia, Rithvik Gottumukkala Team Leader), Dash Doubleday
Domain / Industry:	Cybersecurity and Privacy

1. Business Problem & Target Impact

S-Card would filter messages using a portable personal profile and local AI. Signup would use a random code, without requiring a name or email [4]. Encrypted preferences could move between approved connectors. Provider accounts remain identifiable. The goal is fewer interruptions without hiding bank alerts or verification codes. We will compare personalized and default settings using spam recall, false-positive rates, and latency. On-device, multilingual detection remains a research opportunity [1].

2. Technical Scope & Architecture

Data Sources: Almeida and Hidalgos UCI SMS Spam Collection (2011) and Apache SpamAssassin (20030228); 8,361 cleaned messages. Historical spam labels do not validate phishing detection.

Methodology: TF-IDF/logistic regression, local thresholds and preference rules, grouped splits, and separate SMS/email evaluation. Quarantine would be reversible. Start with a Gmail OAuth test mailbox (gmail.modify); public rollout requires verification [2].

Primary Tech Stack: Planned Python/scikit-learn, ONNX Runtime local inference [3], a local Streamlit demo, PostgreSQL model registry, Docker/AWS EC2, and S3 model releases. Personal messages and feedback would stay local; device-held keys would protect preferences. Further email, mobile, chat, and carrier integrations depend on permitted APIs or partnerships.

3. Project Milestones & Deliverables

Current Progress	Cleaning and profiling exports complete. S-Card features remain proposed.
Oct 28 Preview Deliverable	Two preference profiles, quarantine demonstration, and database schema.
Nov 30 Showcase Deliverable	Local inference, model releases, encrypted preference export, test-mailbox connector, and measured errors/latency.

4. Key Discussion Questions for Industry Mentors

We welcome feedback on two core technical and strategic governance challenges:

1. Technical / Architectural Scaling

How should organizations balance response time, device battery life, data privacy, and cost when deciding whether an AI application should run locally or use cloud-based inference?

2. Business / Governance & Compliance

What level of false positives would a bank consider acceptable before piloting this system with customers, and what safeguards and remediation processes should be in place when errors occur?

References:

1 Hur, M., Seo, S., Hwang, J., & Min, M. (2026, July 16). Are large language models effective for detecting spam messages? International Journal of Information Security, 25, Article 127. https://doi.org/10.1007/s10207-026-01283-5

2 Google. (2026, September 10). Choose Gmail API scopes.

3 ONNX Runtime. (n.d.). Get started with ONNX Runtime Mobile. Retrieved October 15, 2026.

4 Mullvad VPN. (2026, June 17). No-logging of user activity policy (privacy-design inspiration).

Team 7: CoursePilot — Autonomous Academic Degree Navigation & Advisory Assistant

Project Title:	CoursePilot - AI Academic Advising Assistant
Team Members:	Mujtaba Ali Khan, Cayman Powell (Team Leader), Alexis Arenas-Villalba
Domain / Industry:	Educational AI, Academic Advising, NLP

1. Business Problem & Target Impact

Students from UNC Charlotte have to tread through several different sources to gain the understanding of requirements, to compare courses and to plan academic progress. This is a disjointed experience which can lead to misunderstandings and requests for advising that are repeated. CoursePilot is an AI tool that was proposed to be able to access academic authoritative information, give straightforward answers in plain language and augment official advice. Early users are College of Computing and Informatics (CCI) students and advisors. This incorporates measures such as answer correctness with source citations, covering common course questions, time to answer and minimising routine advising requests. Target setting will be carried out through testing.

2. Technical Scope & Architecture

Data Sources: UNC Charlotte syllabus system (about 286 CCI syllabi identified for Fall 2026), course catalog, optional DegreeWorks audits by students, and university policies.

Methodology: Batch collection, PDF/HTML extraction, duplicate detection, clean, validate metadata, planned document chunking, vector embeddings, semantic retrieval and RAG, with catalog year/semester metadata for traceable answers.

Primary Tech Stack: React/Next.js, Python/FastAPI, Amazon RDS (PostgreSQL with pgvector), Amazon S3, Open Source LLMs and/or OpenAI API, & Docker.

3. Project Milestones & Deliverables

Current Progress	Sprint 1 identified data sources, assessed syllabus coverage and quality, duplicate, and inconsistent records, and proposed a data ingestion-to-RAG architecture.
Oct 28 Preview Deliverable	Prepare first interface, first backend-to-data connection, retrieve/query course information (show traceability), discuss remaining work; show Docker architecture.
Nov 30 Showcase Deliverable	Deliver an integrated CoursePilot prototype with CCI-focused document ingestion, semantic retrieval, natural-language answers with source references, and evaluation of accuracy, latency, and usefulness.

4. Key Discussion Questions for Industry Mentors

We welcome feedback on two core technical and strategic governance challenges:

1. Technical / Architectural Scaling

What factors should guide the choice between managed cloud services and containerized deployment, particularly when data persistence, source traceability, scalability, and operational overhead are important requirements?

2. Business / Governance & Compliance

What metrics and safeguards should a university use to evaluate an AI advising assistants impact on student outcomes and advisor efficiency while ensuring that its guidance on degree requirements is accurate, current, and trustworthy?